Ocean University of China

Robotics Integration Group Project

Laboratory Report

Desktop Object Detection and ROS 2 Result Publishing with YOLO11n

Student: Shuyu Li

Student ID: 24020036035

School: College of Computer Science and Technology

Faculty: Faculty of Information Science and Engineering

Program: Computer Science and Technology (Sino-Foreign Cooperative Education), Class of 2024

Course: Robotics Integration Group Project

Instructor: Xiaowei Zhou

University: Ocean University of China

Project Repository:

https://github.com/BBOOOKK/summer_2026

September 2026

Abstract

This experiment implements five-class desktop object detection and ROS 2 result publication using YOLO11n. The current dataset contains 5,000 images and 8,507 bounding boxes: 1,000 files derived from 10 self-collected source photographs and 4,000 Open Images V7 photographs. Parent grouping keeps derivatives of the same photograph within one split. The detector was fine-tuned for 12 epochs from a historical five-class checkpoint whose training history includes mixed-source data. On the fixed 650-image development holdout, mean per-class precision and recall were 0.8385 and 0.7697, with mAP@0.5 of 0.8386 and mAP@0.5:0.95 of 0.5735. A deterministic, class-balanced subset yielded 20/20 correct highest-confidence class decisions at confidence 0.50; separate box-level inspection identified localization errors, missed objects, and a false positive. A desktop frame demonstrated simultaneous laptop, keyboard, and mouse detection. On a Jetson Orin NX, PyTorch FP16 required 25.951 ms per complete `model.predict` call, equivalent to 38.534 FPS. The ROS 2 result topic reached 29.99 Hz, closely matching the camera input rate. These measurements support the specified recognition and real-time criteria under the stated protocols. The implementation provides annotated output, structured records, preserved review cases, and reproducible commands, with dataset provenance and checkpoint identity linked to the project repository.

Keywords: object detection; YOLO11n; desktop objects; data augmentation; Jetson Orin NX; ROS 2

1 Introduction

Desktop manipulation and human-robot interaction require more than a class label. A robot also needs the location of each object so that later modules can associate detections with actions. Mice, keyboards, and laptop computers commonly occur together in a confined workspace. An open laptop introduces nested semantics because the complete device and its visible keyboard are both meaningful targets. Illumination, occlusion, viewing angle, and object pose further alter the observed appearance. A practical detector must therefore process a continuous image stream while returning class labels, bounding boxes, and confidence scores.

YOLO formulates object classification and bounding-box regression in a single forward pass and is representative of one-stage detection[1]. SSD likewise performs multi-scale prediction without an independent region-proposal stage[2]. These designs are suitable for continuous image inference when the target platform has limited computing resources. The present experiment adopts the compact YOLO11n model without altering its architecture.

The work connects dataset construction and detector fine-tuning with camera inference and robot communication. Three experimental questions organize the evaluation:

EQ1. What detection quality does the selected model achieve across the five classes on the fixed 650-image development holdout?

EQ2. Does the implementation demonstrate simultaneous detection of at least two classes and at least 80% correct class decisions on the specified 20-image acceptance sample?

EQ3. Does the deployed Jetson pipeline exceed 5 FPS while displaying detections and publishing results through ROS 2?

The evaluation combines aggregate detection metrics, a fixed class-recognition test, inspection of individual failures, and measurements of the deployed image stream. Dataset provenance and checkpoint history establish how the model was obtained; source-stratified analysis then examines how the holdout composition affects the result. Runtime measurements distinguish network inference, complete model calls, and ROS 2 publication, each of which answers a different question about robot integration.

Accordingly, the principal outcome is an end-to-end implementation covering dataset construction, model fine-tuning, offline evaluation, camera inference, error preservation, and ROS 2 integration. The purpose is not to propose a new neural-network architecture, but to demonstrate that a compact detector can be trained, measured, deployed, and documented consistently for a desktop robotics scenario.

2 Related Work

2.1 One-Stage Object Detection

The original YOLO method predicts bounding boxes and class probabilities directly from the complete image[1]. SSD places default boxes on multiple feature maps so that objects of different scales can be handled at suitable resolutions[2]. Focal Loss addresses dense-detection imbalance through loss weighting by reducing the contribution of easy background examples[3]. These studies established complementary approaches to direct regression, multi-scale prediction, and foreground-background balancing.

Data augmentation can vary the appearance of training samples without modifying the deployed network. YOLOv4 combined color perturbation, geometric transformation, and Mosaic augmentation in its training procedure[4]. In this experiment, offline transformations use a restricted rotation range to remain close to plausible camera observations. Offline, traceable expansion is kept separate from the online augmentation applied by the training framework.

YOLO11 is distributed through the Ultralytics package, which provides consistent interfaces for training, validation, prediction, and export. The nano-scale variant is intended for lightweight inference[5]. This experiment prioritizes compatibility between the macOS training environment and the Jetson CUDA environment, together with sufficiently low model-call latency for camera use. The choice reflects these constraints and does not imply that a smaller model is universally more accurate than larger alternatives.

2.2 Data Resources, Metrics, and Robot Communication

Open Images provides natural-scene images with several forms of annotation, including a large object-detection subset[6]. This experiment uses Open Images V7 files that correspond to the five target categories[7]. Their source identifiers and original split information are preserved when boxes are converted to the local YOLO representation.

Detection quality is reported using precision, recall, mAP@0.5, and mAP@0.5:0.95. The multi-threshold metric follows the COCO convention of averaging performance over several intersection-over-union thresholds[8]; its numerical implementation is provided by the Ultralytics validation interface[9]. A stratified 20-image subset is also reported because the course acceptance criterion is stated as class-recognition accuracy rather than AP.

ROS 2 provides a modular communication architecture for robot systems[10]. Topics exchange sensor data and robot state through a publisher-subscriber interface. Once a topic name and message type are agreed, nodes can communicate without directly invoking one another[11]. The detector can therefore remain independent while downstream software subscribes either to compact structured results or to annotated images.

3 Dataset Construction

3.1 Classes and Annotation Rules

The five classes are mouse, keyboard, laptop, cup, and headphones, with identifiers 0 through 4 in that order. Labels use the Ultralytics YOLO horizontal bounding-box format; each line stores a class identifier, normalized center coordinates, and normalized width and height[12]. In the self-collected annotations, an open laptop receives a complete-device box and its visible keyboard receives a separate keyboard box. Occluded objects are annotated over their visible extent. Open Images annotations retain the provider's box extent after class mapping and coordinate validation.

The nested rule makes the self-collected labels consistent with the intended robot interface. Public images were not exhaustively relabeled, so their annotations cannot be assumed to include every visible nested keyboard. This difference in annotation coverage is relevant when interpreting localization and unmatched detections. The five-class identifier order is nevertheless identical in the dataset configuration, checkpoint, evaluation scripts, and ROS 2 output.

3.2 Source Composition and Offline Expansion

The dataset contains 1,000 self-collected-source images and 4,000 Open Images V7 images[7]. The self-collected-source portion starts from 10 source photographs. Each parent produces one base version normalized to 1280×960 pixels and 99 deterministic augmented versions. Parameters are determined from random seed 20260831 together with the parent filename and variant number. Transformations include rotation within $[-9^\circ, 9^\circ]$, scaling within $[0.88, 1.12]$, horizontal or vertical translation of no more than 7% of the image dimension, horizontal flipping with probability 0.5, and bounded brightness, contrast, blur, and noise perturbations. Bounding boxes are transformed with the image.

The term *1,000 self-collected-source images* describes the material supplied to the learning pipeline. It comprises 10 normalized base images and 990 augmented versions, rather than 1,000 independent captures. Augmentation changes brightness, placement, and local imaging characteristics; the source photographs determine the available rooms, desk arrangements, cameras, and object instances.

The public subset supplements object appearance and background variation. For every Open Images file, the manifest stores the original split, source identifier, and download URL before mapping the class name to the local identifier. If a source box is represented in pixel coordinates, conversion to YOLO format is

$$x_c = \frac{x_{\min} + x_{\max}}{2W}, \quad y_c = \frac{y_{\min} + y_{\max}}{2H}, \quad w = \frac{x_{\max} - x_{\min}}{W}, \quad h = \frac{y_{\max} - y_{\min}}{H}, \tag{1}$$

where W and H are image width and height. Converted coordinates and dimensions are then checked against the valid normalized range.

Figure 1 shows four base images and their deterministic variants.

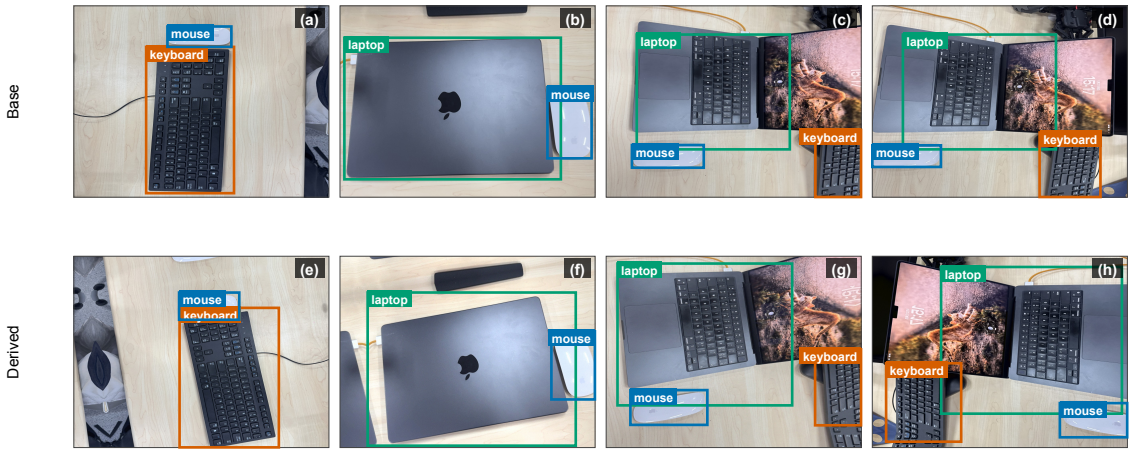


Figure 1. Examples derived from self-collected material. Panels (a)–(d) are size-normalized base versions of four parent photographs; panels (e)–(h) are deterministic augmented versions with consistently transformed boxes.

3.3 Data Splits and Quality Control

The 10 parent photographs are assigned to training, validation, and test directories in an 8:1:1 ratio. Every derivative of a parent remains in the same split. Open Images files retain their original source split and are sampled by stable source-identifier hashes. The final splits contain 4,000, 350, and 650 images (Table 1), corresponding to 3,208, 251, and 551 distinct source identifiers. These identifiers count photographs, not independent scenes. Because the initialization checkpoint had already been evaluated on the retained public test images, the 650-image split is treated throughout as a fixed development holdout; its directory name remains `test` for reproducibility.

Table 1. Dataset splits, image provenance, and bounding-box counts.

Split	Self-collected source	Public	Images	Boxes
Training	800	3,200	4,000	6,773
Validation	100	250	350	627
Holdout (test)	100	550	650	1,107
Total	1,000	4,000	5,000	8,507

Quality control covers image readability, image-label pairing, class identifiers, normalized coordinates, duplicate source identifiers, byte-identical files, and parent-image overlap.

No unreadable image, missing label, byte-identical duplicate, or cross-split derivative of the same parent was found. The current 5,000-image fine-tuning dataset contains zero generated images and zero images of unknown origin. This statement concerns the current dataset; the checkpoint's earlier training history is described in Section 4.2.

Perceptual-hash screening identified four cross-split pairs at Hamming distance 1. They involve derivatives of `IMG_8304`, `IMG_8305`, and `IMG_8306`, which depict closely related desktop arrangements and lie in different splits. Parent grouping therefore prevents exact derivative overlap but does not establish scene independence. The holdout's 100 self-derived images all originate from `IMG_8306` and contribute 300 of its 1,107 boxes (27.10%). Public annotations underwent structural checks and sampled visual inspection; the converted provider boxes remain the evaluation reference.

Table 2 separates box counts by source. Laptop has 2,549 boxes and headphones 871, a ratio of approximately 2.93. Self-collected material covers mouse, keyboard, and laptop; cup and headphones are supported entirely by public images in the current dataset. All 5,000 label files contain at least one target box, so the evaluation does not provide a separate false-alarm rate for target-free scenes.

Table 2. Bounding-box distribution by class and source.

Class	Self-derived	Public	Total	Share (%)
mouse	1,000	724	1,724	20.27
keyboard	595	1,371	1,966	23.11
laptop	700	1,849	2,549	29.96
cup	0	1,397	1,397	16.42
headphones	0	871	871	10.24
Total	2,295	6,212	8,507	100.00

4 System Design and Implementation

4.1 Processing Pipeline

The system comprises dataset construction, model training, unified inference, and ROS 2 publication. Figure 2 summarizes the processing path. The training entry point reads the dataset configuration and produces model weights. The inference entry points accept an image, video, or USB camera stream and store annotated results and tabular records. The ROS 2 node reuses the same model call and threshold configuration before packaging detections as messages.

The modules share one class map and one set of inference parameters but retain separate entry points. This organization prevents camera, file, and ROS 2 modes from implementing different post-processing logic. It also allows a failure to be located at a specific boundary: data reading, model prediction, output serialization, or message publication.

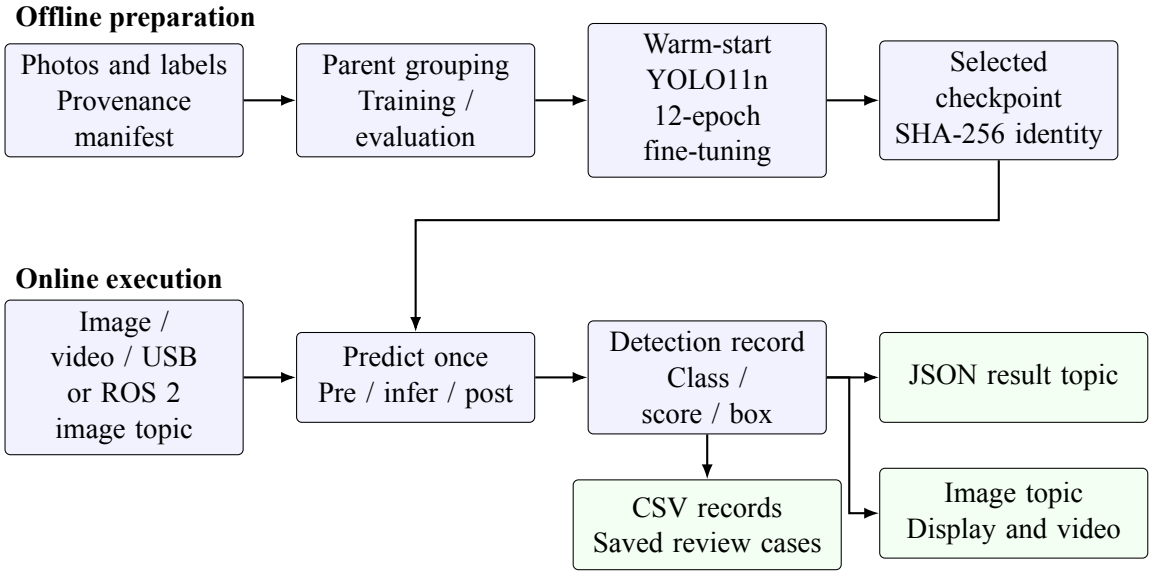


Figure 2. Offline preparation and online execution. A fixed checkpoint connects the two stages; one prediction result supplies structured messages, visual output, and persistent records.

4.2 Model Fine-Tuning

The experiment uses sequential transfer learning. The initial P05 model was trained from `yolo11n.pt` on a 500-image baseline comprising 453 generated images, 37 public images, and 10 local photographs. The subsequent 30-epoch expanded-data run combined that baseline with 4,500 Open Images photographs and produced `models/expanded_v1/best.pt`. This historical checkpoint initializes the present 12-epoch run. Its SHA-256 is `8c9ad192ec37128ad10751bf8d7f58a3ee821c4ae0e47733fe0a8d199212c9df`. Thus, the final weight history includes generated-image supervision even though the current fine-tuning dataset contains none.

All 4,000 public images in the current dataset also occur in the historical expanded dataset with the same split assignments: 3,200 training, 250 validation, and 550 test. The 550 public holdout images were evaluated during the earlier experiment, but the manifests do not assign them to either training split. The current validation and holdout source photographs also retain their earlier validation and test assignments. This lineage supports evaluation on a reused development holdout, without establishing a previously unseen external test.

The current run uses 12 epochs, batch size 32, input size 640, zero data-loader workers, and early-stopping patience 4. Training runs on Apple MPS with random seed 0, deterministic mode requested, and AMP enabled in the configuration. These settings document the run without guaranteeing bitwise equivalence across hardware. Ultralytics selects the optimizer automatically, and `args.yaml` preserves the requested arguments.

Online augmentation includes hue, saturation, and value parameters of 0.015, 0.7, and 0.4; rotation, translation, and scaling parameters are 5 degrees, 0.1, and 0.3. Horizontal flipping has probability 0.5. Mosaic is configured at 0.5 for the first two epochs and disabled for

the final 10; MixUp is disabled. Validation metrics are computed after every epoch, and both the last checkpoint and the checkpoint with the best validation metric are saved.

Table 3 summarizes the parameters required to reproduce the run. The selected acceptance model contains 2,590,815 parameters and occupies 5,482,577 bytes. Its SHA-256 is `c53834aa239974163ef48c566dc3fea04d38de7a1d05292ab5216d5163a09a42`. Fixing the hash prevents a later checkpoint with the same `best.pt` filename from being assigned the reported results.

Table 3. Principal YOLO11n fine-tuning parameters.

Item	Setting
Initialization	`models/expanded_v1/best.pt`; unchanged five-class order
Epochs and early stopping	12 epochs; patience 4
Batch and input	batch 32; 640×640
Training device	Apple M4, MPS; workers 0
Reproducibility	seed 0; deterministic mode requested; AMP enabled in configuration
Optimizer arguments	`optimizer=auto`; the automatic selector ignores the requested `lr0`=0.01; weight decay 0.0005
Geometric augmentation	degrees 5; translate 0.1; scale 0.3; horizontal flip 0.5
Composed augmentation	Mosaic 0.5 in epochs 1–2, then disabled; MixUp 0
Model selection	validation after each epoch; best $AP_{50:95}$ checkpoint for testing

4.3 Inference Records and ROS 2 Interface

Inference uses an input-size parameter of 640. Candidate boxes are filtered by confidence and then processed by non-maximum suppression (NMS). Each stored detection contains a class identifier, class name, confidence, and pixel-coordinate box (x_1, y_1, x_2, y_2). The same data structure drives box drawing and CSV output for image, video, and camera input.

The confidence threshold removes low-score candidates, while NMS suppresses repeated same-class boxes that overlap a higher-confidence box beyond the configured IoU threshold. Acceptance inference uses confidence 0.50 and NMS IoU 0.45 for the 20-image test, Jetson visualization, and saved results. The offline `model.val` evaluation retains the validation interface configuration because AP is obtained from ranked detections rather than from a single presentation threshold.

The detector is also wrapped as a ROS 2 node. In camera mode, the node reads a USB device directly; in topic mode, it subscribes to `/camera/image_raw`. After each prediction, `/object_detection/result` carries JSON in `std_msgs/String`, and `/object_detection/image` carries the annotated frame in `sensor_msgs/Image`.

Separating compact numerical output from the image stream lets a control process and a visualization process subscribe independently[11].

The JSON fields are listed in Table 4. Pixel-coordinate boxes can be overlaid directly on the original frame, while class identifiers support stable program logic and class names simplify terminal inspection. A frame with no accepted detection still publishes its timestamp, frame identifier, FPS value, and an empty detections array. A subscriber can therefore distinguish normal operation with no current target from a publication failure.

Table 4. Principal fields in the ROS 2 structured detection message.

Field	Form	Meaning
`timestamp`	floating point	node-clock time in seconds, assigned after prediction
`frame_id`	integer	processed-frame counter since node startup
`fps`	floating point	exponentially smoothed model-call rate
`detections`	list	all accepted detections in the current frame
`class_id`	integer	class identifier from 0 through 4
`class_name`	string	mouse, keyboard, laptop, cup, or headphones
`confidence`	floating point	class confidence for the current box
`bbox`	object	pixel coordinates x_1, y_1, x_2, y_2

The node performs one model call per frame. JSON, the annotated image, and optional video output are derived from that result, so enabling more than one output does not duplicate neural-network inference. The result-topic queue depth is 10 and the image-topic queue depth is 2. The smaller image queue limits accumulation of large stale frames. Camera and video-writer handles are released explicitly during node shutdown. The JSON timestamp denotes post-prediction node time, rather than camera exposure time. Encoding detections in `String` makes terminal inspection simple, but consumers must validate the JSON fields because the ROS message type does not enforce their schema.

4.4 Jetson Deployment

The acceptance checkpoint is deployed on a Jetson Orin NX. The measured run uses NVIDIA PyTorch 2.5.0, CUDA device 0, and FP16 precision. The network input is 640×640, and confidence and NMS IoU thresholds are 0.50 and 0.45. The USB camera at `/dev/video0` delivers 640×480 frames, which Ultralytics resizes and preprocesses before inference.

TensorRT 10.7.0.23 runtime libraries are present on the device, but the Python environment used for measurement does not contain the `tensorrt` binding. The reported benchmark therefore uses the PyTorch FP16 checkpoint rather than a serialized TensorRT engine. TensorRT engines are commonly built for a target GPU and runtime environment[13, 14].

Recording installed libraries separately from the active backend prevents a software dependency from being misreported as TensorRT performance.

5 Experimental Setup and Evaluation Protocol

5.1 Training and Offline Evaluation Environment

Training and offline evaluation were conducted on macOS with an Apple M4 processor and the MPS backend. Python, PyTorch, and Ultralytics versions were 3.12.4, 2.13.0, and 8.4.133. At epoch 12, validation precision, recall, mAP@0.5, and mAP@0.5:0.95 were 0.7407, 0.7512, 0.7334, and 0.4621. The last value was the highest mAP@0.5:0.95 observed during the 12-epoch run, so this checkpoint was selected for subsequent testing.

5.2 Jetson Environment and Runtime Configuration

Edge-device tests used an NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super. The device operated in `MAXN_SUPER` power mode with dynamic frequency scaling; `jetson_clocks` frequency override was not enabled. Table 5 records the measured configuration.

Table 5. Jetson hardware, software, model, and input configuration.

Field	Measured or verified value
Device and memory	NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super; 16 GB; 16,031,376 kB visible to the system (approximately 15.29 GiB)
System software	JetPack 6.2.1; L4T 36.4.4; CUDA runtime 12.6.77; TensorRT libraries 10.7.0.23; ROS 2 Humble
Power configuration	`nvpmodel`: `MAXN_SUPER`, mode 0; dynamic frequency scaling, `FreqOverride=0`
Model and backend	`models/experiment1_1000self_4000public_v1/best.pt`; YOLO11n; NVIDIA-build PyTorch 2.5.0; CUDA device 0
Precision	FP16
Input	640×640 model input; HDR CAMERA-A USB camera at `/dev/video0`; 640×480 captured frames
Detection thresholds	confidence 0.50; NMS IoU 0.45

5.3 Evaluation Metrics

Let true positives, false positives, and false negatives be TP, FP, and FN. Precision and recall are

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}. \tag{2}$$

The intersection over union between a predicted box B_p and a ground-truth box B_g is

$$\text{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \tag{3}$$

These ratios are computed for each class. The reported mean precision and recall are macro averages over the five classes, rather than ratios formed by pooling all boxes. mAP@0.5 averages class AP at IoU 0.5. mAP@0.5:0.95 additionally averages over IoU thresholds from 0.5 through 0.95 in increments of 0.05 and therefore places a stricter requirement on localization[8, 9]. The validator's summary precision and recall use its confidence-curve operating point; they are distinct from the class decisions made at the fixed deployment confidence of 0.50.

5.4 Test Protocols

First, the 650-image development holdout was evaluated with Ultralytics 8.4.133 using `split=test`, `imgsz=640`, `batch=16`, and `device=mps`. The resolved validator settings use confidence 0.001 as the candidate floor, NMS IoU 0.70, at most 300 detections per image, rectangular batching, class-aware NMS, and no test-time augmentation. Offline evaluation uses FP32. Summary precision and recall are read at the common confidence threshold maximizing the smoothed mean F1 curve across classes, approximately 0.464 in the saved full-holdout curve. The confusion-matrix plot uses a separate confidence threshold of 0.25. These settings distinguish the AP evaluation and diagnostic plot from deployment at confidence 0.50 and NMS IoU 0.45.

A source-stratified evaluation additionally isolates the 550 Open Images photographs and their 807 boxes from this holdout. It uses the same checkpoint, software versions, and metric-affecting validation settings, except that batch size is reduced to 4 for memory-bounded execution. Rectangular batching is retained, so removing images and changing batch size can alter the padded input shapes. Its precision and recall use the subset's own common F1 operating point, 0.4454. The 550 source identifiers are unique, have no overlap with the 37 public images in the historical baseline, and all remained in the initialization dataset's test split. This analysis excludes self-derived scenes but still reuses development data; it is not an additional independent test.

Second, a 20-image class-decision subset was formed from the public portion of the same holdout. For each class, the first four Open Images files in lexicographic manifest order with exactly one reference box were selected; the rule did not consult per-image predictions. All 20 are real public photographs. Inference used MPS, confidence 0.50, and NMS IoU 0.45. A sample was correct when its highest-confidence predicted class matched the manifest class; no detection or a different top class was an error. Lower-ranked detections and bounding-box IoU do not enter this decision. The image count is 20, with one annotated target per image. This

protocol is a fixed photographic acceptance test, not a record of 20 newly presented physical objects. It remains descriptive because the photographs belong to the existing holdout, and its class-only decision is distinct from IoU-matched AP.

Third, real-time performance was measured on the Jetson Orin NX with the USB camera. Timing began after camera acquisition. The first 20 frames were used for warm-up and the next 200 for measurement. Each interval covered Ultralytics preprocessing, GPU inference, and post-processing within `model.predict`; it excluded camera reading, box drawing, video or log storage, and ROS 2 publication. The model used PyTorch FP16 at input size 640 with confidence 0.50 and NMS IoU 0.45.

For timed call durations t_i in milliseconds, the benchmark calculates

$$\bar{t} = \frac{1}{N}\sum_{i=1}^{N} t_i, \qquad F_{\text{call}} = \frac{1000}{\bar{t}}, \qquad F_{\text{min}} = \frac{1000}{\max_i t_i}, \quad N = 200. \tag{4}$$

The reported mean FPS is therefore the reciprocal of mean latency, not the arithmetic mean of individual frame rates. The minimum is the lowest observed call rate within this run.

Finally, ROS 2 communication performance was measured with `ros2 topic hz` on `/object_detection/result`. The sliding window contained 200 messages and the observation interval was 15 s. This frequency represents messages received by the subscriber and differs from the exponentially smoothed model-call FPS embedded in the JSON. The aggregate benchmark CSV is retained on the Jetson at `outputs/benchmark/jetson_orin_nx_camera.csv`. The script writes one summary row; it does not export a per-frame latency series.

5.5 Reproduction Materials and Result Attribution

Key artifacts are stored by experiment stage, as summarized in Table 6. The dataset configuration defines paths and class order; the manifest records per-image provenance; `args.yaml` fixes training parameters; evaluation JSON and CSV files store measurements; and the review directory preserves low-confidence or failed examples produced by the test routine. The reported 650-image metrics and 20-image results are taken from the directory named for the current experiment rather than from earlier generic outputs retained elsewhere in the repository.

This structure addresses a common ambiguity during iterative experiments: identical filenames associated with different checkpoints. A reported value is attributed only after identifying model hash, data version, thresholds, and execution device. The Jetson timing boundary is also fixed by code. Camera acquisition precedes the timer, while drawing and ROS 2 publication follow it. Thus 25.951 ms is correctly described as a complete `model.predict` call, not as latency from camera exposure to subscriber display.

Table 6. Principal reproduction materials and their roles.

Material	Repository location or record
Project repository	https://github.com/BBOOOKK/summer_2026
Dataset configuration	`datasets/experiment1_1000self_4000public_v1/dataset.yaml`
Provenance manifest	`datasets/experiment1_1000self_4000public_v1/manifest.csv`
Provenance audit	`datasets/experiment1_1000self_4000public_v1/qa/provenance_audit.json`
Training configuration	`outputs/training/experiment1_1000self_4000public_v1/args.yaml` and `results.csv`
Holdout metrics	`outputs/evaluation/experiment1_1000self_4000public_v1_test/evaluation_results.json`
Public-source analysis	`outputs/evaluation/experiment1_1000self_4000public_v1_public_holdout/`
20-image selection and results	`configs/test_samples.csv` and `20_object_test_results.csv` in the experiment evaluation directory
Low-confidence review cases	`outputs/errors/experiment1_1000self_4000public_v1_real20/`
Inspected box-level errors	`outputs/errors/experiment1_1000self_4000public_v1_holdout/`
Jetson measurement record	`docs/JETSON_MEASUREMENT.md`; summary CSV retained on the Jetson
Acceptance checkpoint	`models/experiment1_1000self_4000public_v1/best.pt` and its SHA-256
Initialization checkpoint	`models/expanded_v1/best.pt`
Program and ROS 2 node	`scripts/` and `ros2_ws/src/object_detector/`
Result video	`outputs/videos/jetson_live_detection_20260905_193850.mp4`
Running instructions	`docs/RUN_GUIDE.md`, `docs/JETSON_DEPLOYMENT.md`, and `docs/ROS2_GUIDE.md`
English report	`docs/ieee_report/main.tex` and `output/pdf/desktop_object_detection_experiment_report.pdf`

6 Results and Analysis

6.1 Training Behavior and Overall Detection Results

Figure 3 shows the 12-epoch run. All three training losses decrease overall, whereas validation losses fluctuate. Validation mAP@0.5 and mAP@0.5:0.95 reach 0.7334 and 0.4621 at epoch 12, and no clear performance plateau is observed within the scheduled training interval.

From epoch 1 to epoch 12, training box loss decreases from 0.9981 to 0.7168, classification loss from 0.9210 to 0.5428, and distribution focal loss from 1.3143 to 1.0812. Over the same interval, validation box loss and classification loss decrease from 1.4159 and 1.2833 to 1.1461 and 1.0796, whereas validation distribution focal loss changes from 1.7542 to 1.7939 with repeated fluctuations. Training-set convergence is therefore more regular than validation localization behavior.

Validation mAP@0.5 increases from 0.6760 at epoch 1 to 0.7334 at epoch 12, while mAP@0.5:0.95 increases from 0.3804 to 0.4621. The final three mAP@0.5:0.95 values are 0.4476, 0.4450, and 0.4621. The selected checkpoint is the best observed within the fixed training budget; its occurrence at the final epoch does not establish a performance plateau. Total training time was 3,467.77 s, approximately 57.8 min. These results describe a single run and do not estimate variation across random seeds.

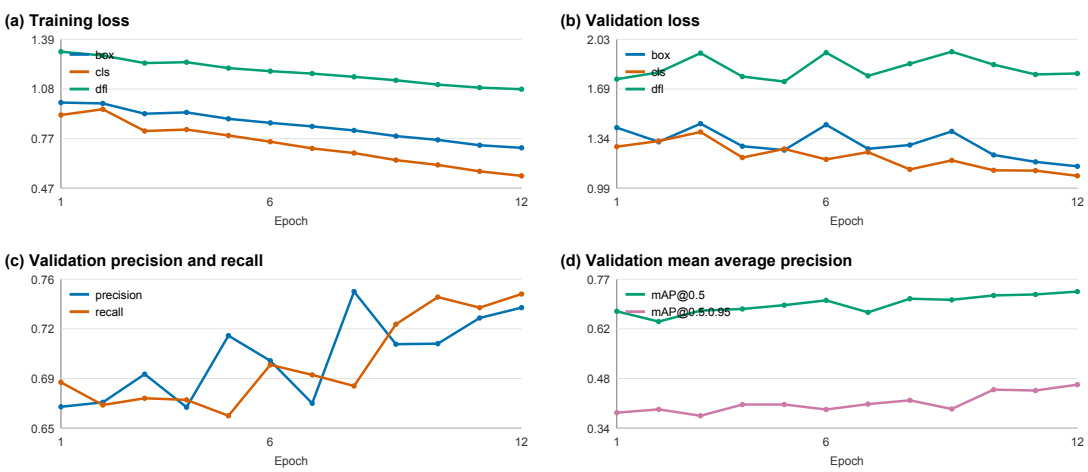


Figure 3. YOLO11n fine-tuning history: (a) training losses; (b) validation losses; (c) validation precision and recall; and (d) validation mean average precision.

Detection results on the 650-image development holdout are reported in Table 7. Mean per-class precision is 0.8385 and recall is 0.7697. At the validator's operating point, accepted predictions are more precise than the model's coverage of annotated objects. mAP@0.5 is 0.8386 and mAP@0.5:0.95 is 0.5735. Their difference of 0.2651 indicates a substantial loss of performance under stricter box-overlap requirements.

Table 7. Per-class and macro-averaged metrics on the fixed development holdout. Δ is AP_{50} minus $AP_{50:95}$.

Class	P	R	AP_{50}	$AP_{50:95}$	Δ
mouse	0.9325	0.9101	0.9501	0.7546	0.1955
keyboard	0.8346	0.8469	0.9012	0.6361	0.2651
laptop	0.9121	0.7529	0.8640	0.4599	0.4041
cup	0.9091	0.7409	0.8571	0.6751	0.1820
headphones	0.6041	0.5978	0.6203	0.3419	0.2784
Macro mean	0.8385	0.7697	0.8386	0.5735	0.2651

6.2 Class Differences and Error Patterns

Mouse has the strongest result, with $AP_{50} = 0.9501$ and recall 0.9101. Keyboard includes both standalone keyboards and visible laptop-keyboard regions, yet reaches $AP_{50} = 0.9012$. Laptop precision is 0.9121, but recall is 0.7529 and $AP_{50:95}$ decreases to 0.4599, indicating weaker target coverage and stricter localization.

Headphones has the lowest precision, recall, and AP_{50}, at 0.6041, 0.5978, and 0.6203. It also has the fewest boxes and appears in open, folded, and partially occluded configurations. These observations show that lower sample count and weaker performance co-occur; they do not establish a direct causal relationship.

Cup precision is 0.9091 while recall is 0.7409. Accepted cup predictions are therefore usually correct, but a portion of ground-truth objects remain unmatched. Keyboard precision and recall are closer, at 0.8346 and 0.8469. These metric combinations describe different failure tendencies: lower precision is associated with more false positives, lower recall with more missed targets, and a wide AP_{50} to $AP_{50:95}$ gap with reduced localization stability under strict overlap criteria.

For laptop, AP_{50} is 0.8640 but $AP_{50:95}$ is 0.4599, a difference of 0.4041. An open laptop contains nested complete-device and visible-keyboard boxes, whereas a closed laptop presents a wide, low silhouette. These annotation semantics and appearance changes may contribute to localization complexity. The interpretation is consistent with inspected samples, but pose-specific AP was not computed and no direct causal claim is made.

The normalized confusion matrix is shown in Figure 4 at its diagnostic confidence threshold of 0.25. With columns normalized by true class, diagonal values for mouse, keyboard, laptop, cup, and headphones are 0.93, 0.89, 0.82, 0.78, and 0.67. In the background row, cup and headphones reach 0.22 and 0.32, indicating more unmatched ground-truth targets for these classes. Explicit class-to-class off-diagonal confusion is comparatively limited, consistent with the lower recall of cup and headphones.

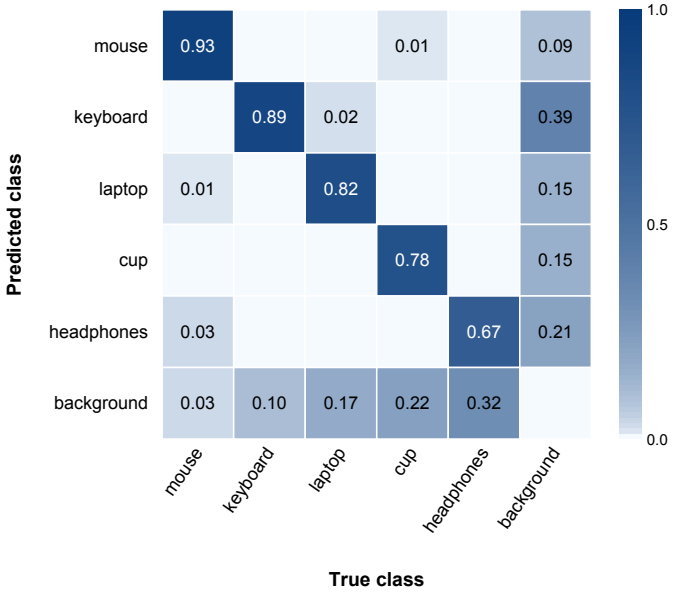


Figure 4. Normalized confusion matrix on the 650-image development holdout. Columns are true classes and rows are predicted classes.

The 20-image script preserves a sample when its correct top-class decision has confidence below the review threshold of 0.60. Three such cases occur in the deterministic subset: two keyboards and one laptop. They are retained as inspectable low-confidence examples, not as estimates of failure frequency. Samples with no detection or an incorrect top class are also retained when present; lower-ranked false positives require separate box-level inspection.

6.3 Twenty-Image Class Decisions and Multi-Class Output

For the deterministic 20-image subset, the highest-confidence predicted class matches the reference class for all 20 photographs, giving 100% class-decision accuracy and a margin of 20 percentage points above the 80% criterion. These images are part of the 650-image development holdout, so the result describes the fixed photographic acceptance sample.

Per-class results are shown in Table 8. Every class achieves 4/4. Mean confidence is calculated over the four accepted class decisions for each category and is included to describe the score range, not as a calibrated probability or global reliability estimate. Keyboard has the lowest mean confidence, 0.6543, and includes the two lowest scores, 0.5083 and 0.5654; both remain above the configured confidence threshold.

To examine simultaneous multi-class output, the current checkpoint was applied to `self_IMG_8306_v000.jpg`, a base image in the self-collected-source portion of the holdout. Inference on MPS used confidence 0.50 and NMS IoU 0.45. Laptop, keyboard, and mouse were detected in the same frame with confidence 0.9233, 0.9184, and 0.8631 (Figure 5). This is a functional demonstration on a familiar desktop arrangement.

Table 8. Per-class decisions on the 20 single-reference-box images.

Class	Correct/total	Accuracy (%)	Mean confidence[a]	Range[a]
mouse	4/4	100	0.9214	0.8609–0.9642
keyboard	4/4	100	0.6543	0.5083–0.8501
laptop	4/4	100	0.7774	0.5904–0.9089
cup	4/4	100	0.8881	0.7531–0.9595
headphones	4/4	100	0.8038	0.7140–0.8703
Overall	20/20	100	–	–

[a]Calculated over the four accepted class decisions in each row.

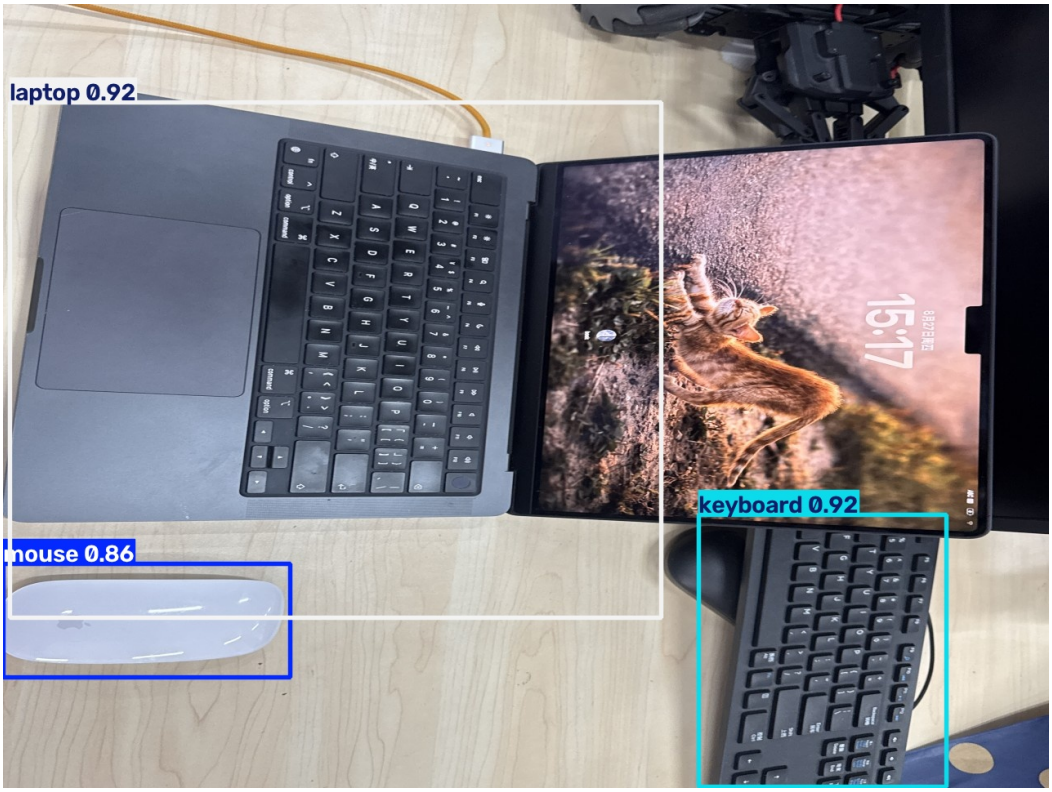


Figure 5. Functional multi-class demonstration on a base holdout image derived from self-collected material. Laptop, keyboard, and mouse are detected simultaneously with bounding boxes and confidence values.

6.4 Jetson Real-Time Detection and ROS 2 Publication

After 20 warm-up frames, the Jetson benchmark measured 200 camera frames. Mean `model.predict` call time was 25.951 ms, corresponding to 38.534 FPS, and the minimum per-frame call rate was 37.952 FPS. Ultralytics reported mean network inference time of 20.575 ms and maximum network inference time of 20.896 ms. The 5.376 ms difference between mean model-call time and mean network time contains preprocessing, post-processing, and prediction-interface overhead. Mean model-call throughput is 7.71 times the 5 FPS acceptance threshold, and the minimum observed call rate also exceeds the threshold.

Figure 6 contains 32 chronologically selected frames from a recording of the browser-displayed Jetson detection stream. The sequence covers 0.25–15.60 s and is arranged from left to right and top to bottom. As the view moves across the desktop, laptop, keyboard, and mouse labels, confidence values, and boxes appear continuously. FPS displayed in the upper-left corner is the program's exponentially smoothed model-call rate for runtime observation; the formal performance value remains the fixed warm-up and measurement benchmark.

The subscriber observed `/object_detection/result` at 29.99 Hz over 15 s, effectively matching the approximately 30 FPS camera input. This demonstrates publication at the source rate. The separate model-call benchmark indicates processing capacity above that input rate; the measurements do not isolate the individual costs of acquisition, serialization, drawing, and image conversion. Both model-call throughput and result-topic frequency exceed the 5 FPS criterion in the measured configuration. Table 9 connects each acceptance item to its evidence and decision rule.

Table 9. Acceptance requirements and corresponding evidence. Numerical decisions apply to the protocols defined in this report.

Requirement	Observation and evidence	Decision
At least two classes simultaneously	Laptop, keyboard, and mouse in one frame; Figure 5	Demonstrated: 3 classes
At least 80% correct on 20 targets	20/20 top-class decisions on the fixed photographic sample; Table 8	PASS for this protocol; +20 percentage points
Jetson detection at ≥ 5 FPS	38.534 FPS mean and 37.952 FPS minimum call rate in 200 timed frames	PASS; mean is $7.71\times$ the criterion
Real-time labels, boxes, and scores	Annotated stream sequence and continuous recording; Figure 6	Demonstrated
ROS 2 result publication	`/object_detection/result`: 29.99 Hz at the subscriber (15 s; window 200)	Measured at approximately camera rate
Save results and typical failures	Detection CSV, video, low-confidence cases, and inspected holdout errors; Section 7.1	Artifacts retained
Six submission materials	Dataset, checkpoint, program, video, instructions, and report; Table 6	Supplied in the submission archive

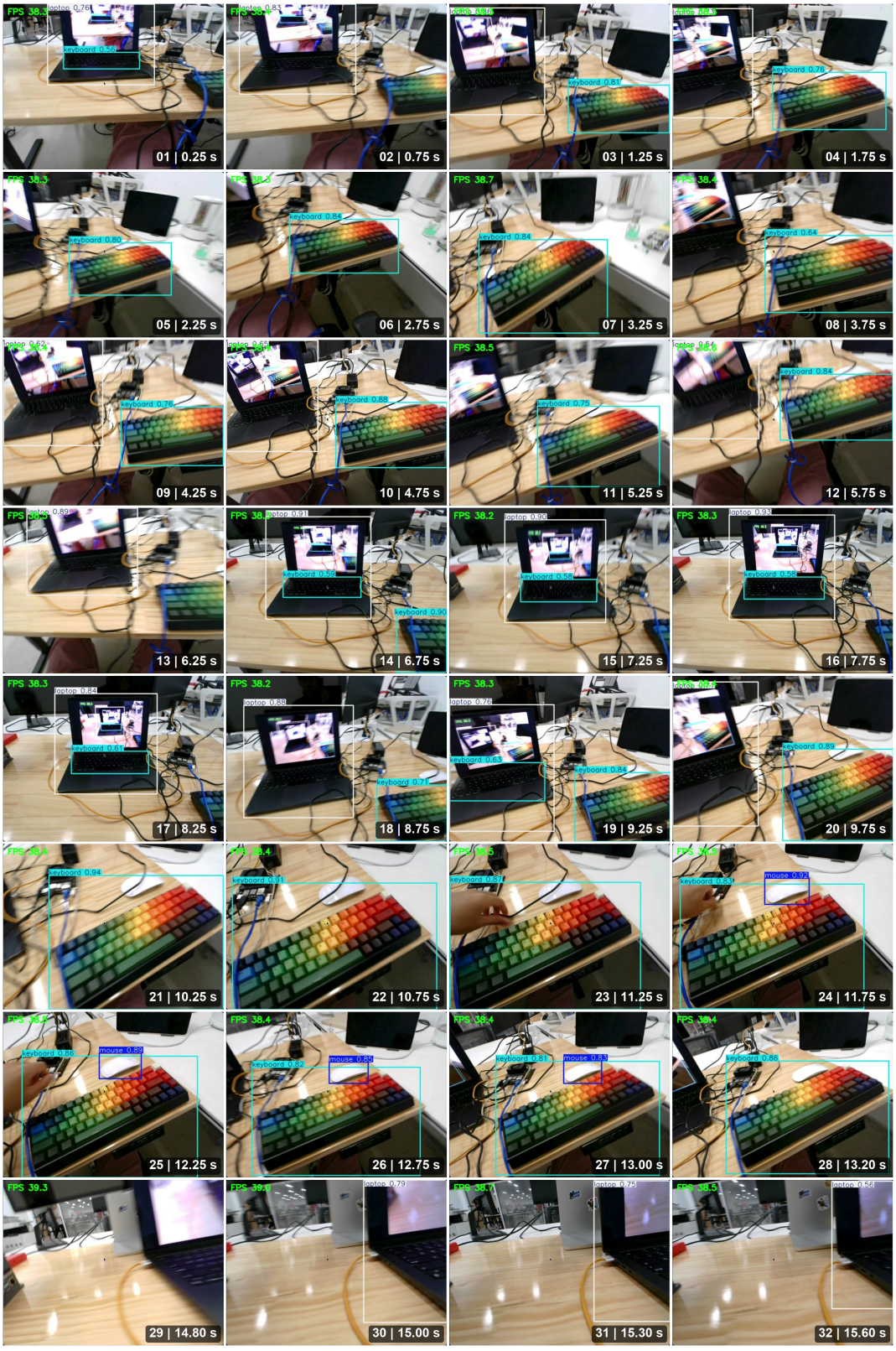


Figure 6. Thirty-two frames from the Jetson real-time detection recording, arranged chronologically from 0.25 to 15.60 s. Each panel retains class labels, confidence values, bounding boxes, and the program-displayed exponentially smoothed model-call FPS.

7 Discussion

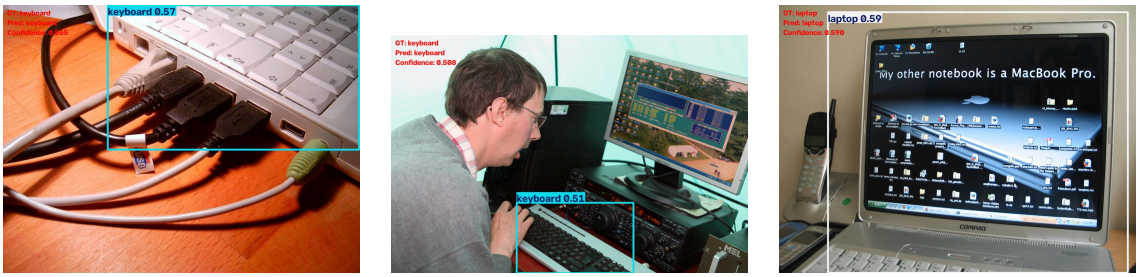
7.1 Recognition, Localization, and Threshold Sensitivity

The 20-image class result and the detection metrics answer different questions. AP evaluates ranked predictions against box annotations, whereas the acceptance sample checks only the highest-confidence class. A correct class decision can therefore coexist with an inaccurate bounding box. The saved case analysis below examines this distinction directly and supplements the class-level patterns in Table 7.

The lowest accepted score in the 20-image sample is 0.5083, only 0.0083 above the configured threshold. Table 10 applies stricter score cutoffs to the saved top-class decisions. At 0.60, 17 remain accepted; at 0.70, 16 remain, exactly meeting the 80% criterion. At 0.75, only 14 remain. This is a retrospective score-filtering analysis of fixed predictions, not a fresh detection run or a basis for choosing a new threshold. It shows that the perfect class score at 0.50 includes several marginal decisions.

Table 10. Retrospective score filtering of the 20 saved correct top-class decisions. Rejected decisions count as unsuccessful samples.

Score cutoff	Accepted correct	Rejected	Success rate (%)
0.50	20	0	100
0.60	17	3	85
0.70	16	4	80
0.75	14	6	70



(a) Keyboard: 0.5654 (b) Keyboard: 0.5083 (c) Laptop: 0.5904

Figure 7. Correct class decisions retained by the low-confidence review policy. All three scores exceed the inference threshold of 0.50 but fall below the review threshold of 0.60. These cases illustrate threshold sensitivity and are not misclassification examples.

Three additional inspections use the same checkpoint with single-image MPS inference at confidence 0.50 and NMS IoU 0.45 (Figure 8). The cases were selected purposefully from the public holdout to examine the weakest class. Their records preserve source identifiers, input and label hashes, all reference and predicted boxes, confidence values, and pairwise IoU. They do not modify the aggregate AP result or estimate the prevalence of each error type.

In case (a), the model returns no accepted detection for a visible pair of headphones. In case (b), it predicts headphones with confidence 0.7140 but covers only part of the provider's reference extent, producing IoU 0.4682. This image is also `real_17` in the 20-image sample: its class decision is correct, yet it fails an IoU 0.50 box-matching rule. In case (c), a correct headphones prediction (confidence 0.6108, IoU 0.7689) accompanies a spurious cup prediction with higher confidence, 0.8460. No cup is visible in the source. Class-aware NMS permits overlapping boxes of different classes to remain, so it does not resolve this semantic conflict. These examples show why classification success, localization quality, and false-positive control require separate inspection.

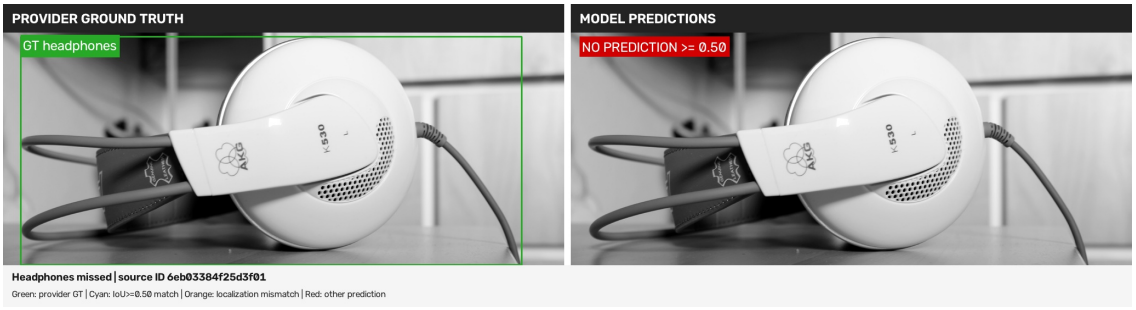
7.2 Source Composition and Scope of the Detection Results

Table 11 reports the public-only analysis. Across 550 photographs and 807 boxes, macro precision and recall are 0.7927 and 0.7114, with mAP@0.5 of 0.7762 and mAP@0.5:0.95 of 0.5593. Relative to the mixed 650-image holdout, the two AP measures are lower by 0.0623 and 0.0142. The larger numerical decrease occurs at IoU 0.50. Because the cohorts and rectangular-batch input shapes differ and AP is nonlinear, neither difference estimates the contribution of the 100 self-derived images.

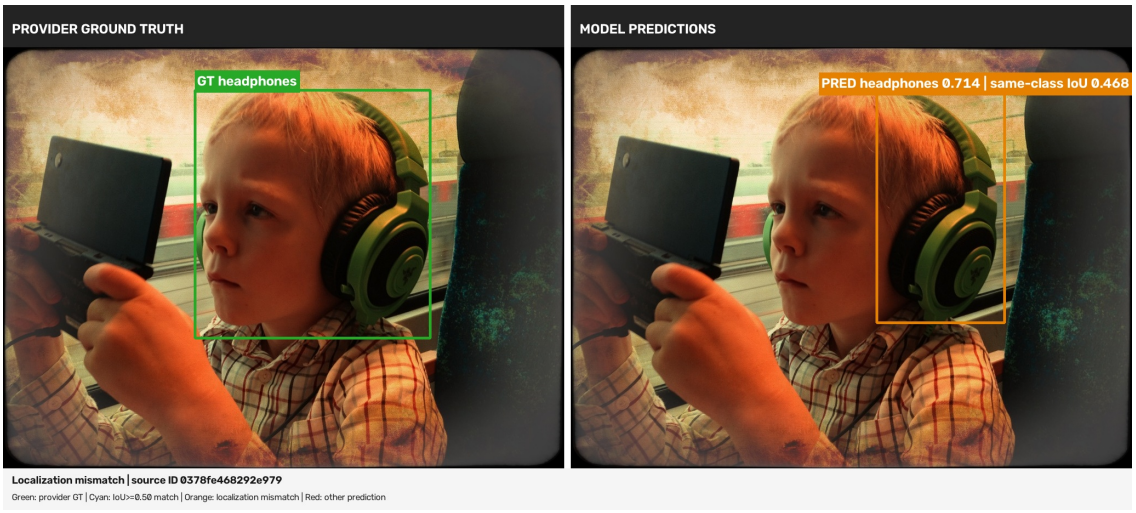
Table 11. Source-stratified evaluation on the 550 public holdout images. Precision and recall use each cohort's shared F1 operating point: 0.4454 for public images and approximately 0.464 for the mixed holdout. The last row reproduces the mixed-holdout macro result for context.

Class or cohort	Boxes	Precision	Recall	AP_{50}	$AP_{50:95}$
mouse	78	0.8664	0.8312	0.8818	0.6955
keyboard	174	0.7454	0.7816	0.7843	0.5370
laptop	162	0.8370	0.6023	0.7360	0.5425
cup	301	0.9105	0.7440	0.8564	0.6732
headphones	92	0.6042	0.5978	0.6227	0.3484
Public 550 (macro)	807	0.7927	0.7114	0.7762	0.5593
Mixed 650 (macro)	1,107	0.8385	0.7697	0.8386	0.5735

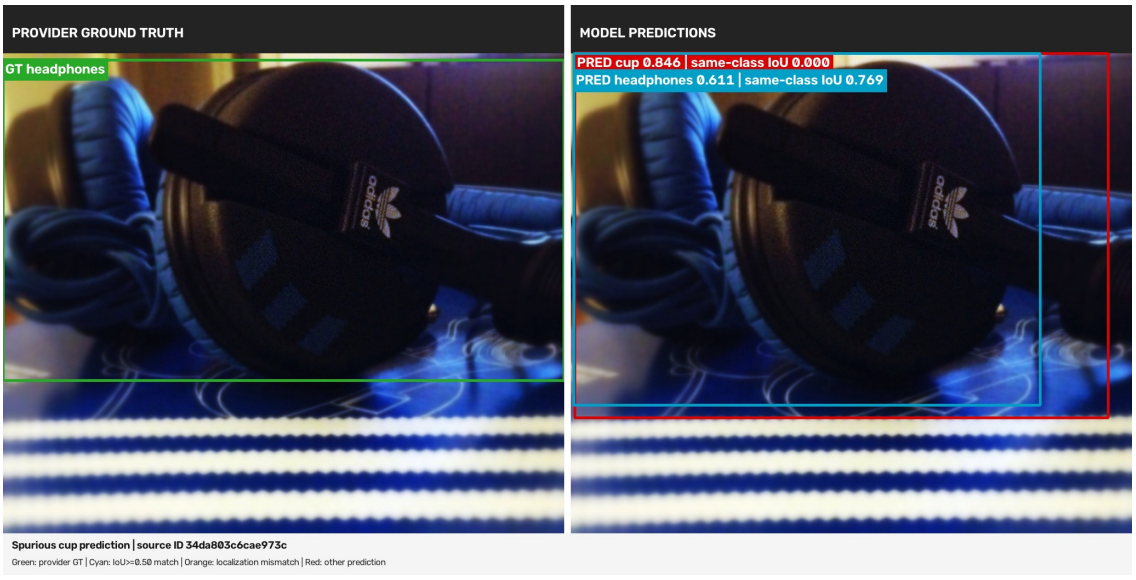
The class pattern is more informative than the aggregate difference alone. Mouse, keyboard, and laptop all show lower AP_{50} on public images, whereas cup and headphones remain close to their mixed-holdout values. This is consistent with the latter two classes having no self-derived reference boxes. Small differences can still occur because rectangular batch composition changes the padded inputs; the public-only run is not a subtraction of saved full-holdout predictions. Laptop $AP_{50:95}$ rises from 0.4599 to 0.5425 even as its AP_{50} falls. For laptop, the cohort comparison therefore has no uniformly favorable direction across overlap criteria. The analysis describes source-associated variation, not a difficulty-matched comparison or an ablation of training data.



(a) Missed detection: source `6eb03384f25d3f01`.



(b) Localization mismatch: source `0378fe468292e979`.



(c) Spurious cup prediction: source `34da803c6cae973c`.

Figure 8. Purposefully selected box-level failures. Each row compares the provider annotation (left) with predictions (right) on the same image. Green denotes reference boxes; orange marks localization mismatch, red an unmatched prediction, and cyan a class-correct match at IoU ≥ 0.50. The source pixels are preserved beneath the annotations.

The holdout AP values also exceed their final-epoch validation counterparts by 0.1052 at IoU 0.5 and by 0.1114 across IoU thresholds. Those splits differ in source proportions and object composition: self-derived images account for 28.57% of validation images and 15.38% of holdout images. Their AP difference therefore cannot be interpreted as a performance gain caused by moving from validation to testing.

The limited number of source photographs is the main constraint on conclusions about new desktop scenes. The holdout contains only one self-collected parent, with nearby arrangements represented in training and validation. Augmentations provide useful appearance variation but remain correlated observations. Public images extend category coverage, especially for cup and headphones, while their inherited annotations introduce uncertainty about box extent and exhaustive target coverage. Together with prior access to the public holdout during model development, these properties support a measured account of performance on the stated data rather than an estimate of deployment accuracy across unseen environments.

7.3 Runtime Interpretation and Robot Integration

Jetson model-call throughput is 7.71 times the 5 FPS criterion, and result-topic frequency is approximately 6.00 times that criterion. The 29.99 Hz result is consistent with the 30 FPS source limit, so the difference from 38.534 FPS does not quantify a processing penalty. The measured durations describe complete prediction calls, while topic frequency describes subscriber arrivals. Neither is a distribution of latency from camera exposure to downstream action. The evidence is specific to the recorded Orin NX configuration, FP16 backend, camera, and detection thresholds.

The multi-class image demonstrates concurrent laptop, keyboard, and mouse output, and the 32-frame sequence shows annotated detections as the camera view changes. The original recording preserves the continuous sequence for inspection. These visual results complement the numerical measurements but do not independently determine aggregate accuracy or minimum throughput. From an integration perspective, reusing one prediction result for messages, visualization, and storage keeps the outputs consistent and avoids repeated network execution within a frame.

8 Conclusion

This experiment completed a five-class desktop perception pipeline from data preparation and YOLO11n fine-tuning to Jetson inference and ROS 2 publication. For EQ1, the selected checkpoint achieved mAP@0.5 of 0.8386 and mAP@0.5:0.95 of 0.5735 on the fixed 650-image development holdout. Per-class analysis identified headphones as the weakest category and laptop as the most sensitive to stricter localization requirements. For EQ2, the fixed photographic acceptance sample produced 20/20 correct top-class decisions at confidence 0.50, and a desktop frame demonstrated three classes simultaneously. For EQ3, Jetson model-call throughput reached 38.534 FPS and the subscriber observed 29.99 result messages per second, both above the 5 FPS criterion.

The principal outcome is a reproducible experimental implementation with a consistent class map, identified checkpoints, explicit measurement protocols, saved detection and review records, and documented execution commands. The results establish the reported functionality and performance in the measured setting. Their interpretation retains the constraints imposed by sequential fine-tuning, reused development data, correlated self-collected scenes, and provider annotations. The accompanying repository and submission archive connect these conclusions to the dataset, model, program, video, running instructions, and report.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Computer Vision – ECCV 2016*, ser. Lecture Notes in Computer Science, vol. 9905, 2016, pp. 21–37.

[3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.

[4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[5] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024, accessed: Sep. 5, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, T. Duerig, and V. Ferrari, "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," *International Journal of Computer Vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[7] Google, "Open Images Dataset V7 and extensions," 2022, accessed: Sep. 5, 2026. [Online]. Available: https://storage.googleapis.com/openimages/web/index.html

[8] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, 2014, pp. 740–755.

[9] Ultralytics, "Model validation with Ultralytics YOLO," 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.ultralytics.com/modes/val/

[10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[11] Open Robotics, "ROS 2 Humble documentation: Topics," 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html

[12] Ultralytics, "Object detection datasets overview," 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.ultralytics.com/datasets/detect/

[13] NVIDIA Corporation, "How TensorRT works," 2026, TensorRT 11.2.1 documentation; Accessed: Sep. 5, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html

[14] ——, "TensorRT engine compatibility," 2026, TensorRT 11.2.1 documentation; Accessed: Sep. 5, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/engine-compatibility.html